

Var $O(N)$

Esercizi con variabili di complessità lineare

Media aritmetica

```
int main() {
    int n, m, p, cont, somma=0;
    printf("quanti valori vuoi inserire");
    scanf("%d", &n);
    for(cont=0; cont<n; cont=cont+1) {
        printf("quali sono i valori");
        scanf("%d", &m);
        somma=somma+m;
    }
    printf("media dei valori di m = M %f", (somma*1.0f)/n);
}
```

Divisione

a sottrazioni successive

```
void main(){
    int n , d,c=0;
    printf("inserisci un dividendo: \n");
    scanf("%d",&n);
    printf("inserisci un divisore:\n");
    scanf("%d",&d);
    while (n>=d) {
        n=n-d;
        c++;
    }
    printf("Quoto: %d Resto: %d",c,n);
}
```

Prodotto

con somme successive

```
int main() {
    int x;
```

```

int y;
int accumulatore=0;
int cont=0;
printf("inserisci valore di x");
scanf("%d",&x);
printf("inserisci valore di y");
scanf("%d",&y);

for(cont=0; cont<y ; cont++)
    accumulatore+=x;
printf("moltiplicazione tra x,y=r %d",accumulatore);
}

```

Elevamento a potenza

con prodotti successivi

```

int main() {
    int x;
    int y;
    int accumulatore=1;
    int cont=0;
    printf("inserisci valore di x:");
    scanf("%d",&x);
    printf("inserisci valore di y:");
    scanf("%d",&y);

    for(cont=0; cont<y ; cont++)
        accumulatore*=x;
    printf("potenza x elevato alla y=r %d",accumulatore);

}

```

Radice quadrata

calcolata per difetto

Estrarre la radice quadrata intera per difetto di un numero N, trovando il massimo numero R che elevato al quadrato non superi N

```

int main(){
    int N,R;
    printf("inserisci un numero intero N");
    scanf("%d",&N);
}

```

```

    if(N<0){
        printf("il numero non deve essere negativo,\n ");
        return 0;
    }
    R=0;
    while(R*R<=N){
        R++;
    }
    R--;
    printf("la radice quadrata intera per difetto %d e %d,\n",N,R);
}

```

Calcolo interesse con interesse composto

Data una quota di capitale, con un tasso di interesse prefissato, accumulo l'interesse al capitale.

- Dato un numero di anni a quanto ammonta il capitale?

```

//Calcolo prestito con interesse composto
void main{
    int anni, cont=1;
    float capitale, tasso=0.1f;
    printf("Capitale iniziale \n -->");
    scanf("%f",&capitale);
    printf("Dopo quanti anni ?\n-->");
    scanf("%d",&anni);
    while(cont<=anni){
        capitale += (capitale*tasso);
        cont++;
    }
    printf("Il saldo finale ammonta a : %f",capitale);
    return;
}

```

- Dopo quanti anni ho raddoppiato il capitale?

```

int main() {
    float tas, cap;
    printf ("dichiara il tasso");
    scanf("%f", &tas);
    printf("dichiara il capitale");
    scanf("%f", &cap);
    float capfin=cap;
    int i=0;
    while (capfin<cap*2) {
        printf("capitale %f\n", capfin);
        capfin+=capfin*tas;
        i++;
    }
}

```

```

}
printf("il numero di anni è:%d cpn capitale %f", i, capfin);
return 0;
}

```

Conversione numero decimale intero in binario

- Versione semplificata con il risultato presentato con le cifre in ordine inverso

```

int main() {
    int num, resto;

    printf("inserisci un num in base 10: ");
    scanf("%d", &num);
    while (num>0){
        resto = num%2;
        printf("%d",resto );
        num = num/2;
    }
    return 0;
}

```

- Questa soluzione produce il risultato in forma binaria bit con le cifre in ordine corretto. Si accumulano i valori delle cifre all'interno di un uint a 64bit. Ad ogni estrazione di una nuova cifra si assegna una potenza di 10 aggiungendolo alla somma.

Es.: Convertire il numero 19

		Somma=0
19 1	1 * 1	Somma=0+1= 1
9 1	1 * 10	Somma=1+10= 11
4 0	0 * 100	Somma=11+0= 11
2 0	0 * 100	Somma=11+0= 11
1 1	1 * 10000	Somma=11+10000= 10011

```

int main(){
    int x;
    int resto ;
    uint64_t somma=0 ,pot=1 ;

    printf("inserisci il valore di x ");
    scanf("%d",&x);
}

```

```
while(x>0){
    resto=x%2;
    x=x/2;
    somma+=resto*pot ;
    pot*=10 ;
}
printf("\nrisultato:%ld" , somma );
return 0;
}
```

Conversione numero binario in decimale

```
int main(void) {
    int n,acc;
    while (1) {
        printf("inserisci numero");
        scanf("%d", &n);
        if (n < 0)
            break;
        acc=acc*2+n;
    }
    printf("%d", acc);
    return 0;
}
```

Potenze di 2

Scrivere un programma che visualizza tutti i numeri interi che sono potenze di 2 inferiori ad un numero dato (ad esempio se viene fornito in input 50 mostra 1, 2, 4, 8, 16, 32)

```
int main() {
    printf("inserire un numero range");
    int a;
    int cont = 1;
    scanf("%d", &a);
    while (cont <= a) {
        printf("%d ", cont);
        cont*=2;
    }
}
```

Conta numeri positivi pari

Scrivere un programma che legge da tastiera degli interi e che conta il numero di valori pari positivi. Alla lettura di un valore negativo smette di contare e stampa il conteggio ottenuto

```
int main()
{
    int num, conta=0;
    do {
        printf("inserisci un num:\n");
        scanf("%d", &num);
        if(num%2==0&&num>=0)
            conta++; //conta+=1; conta=conta+1;
    } while(num>=0);

    printf("i numeri pari positivi sono %d", conta);
    return 0;
}
```

mcm

```
int main() {
    int a, b, t=1;
    printf("Inserisci il primo numero: ");
    scanf("%d", &a);
    printf("Inserisci il secondo numero: ");
    scanf("%d", &b);
    while ((a*t)%b!=0) {
        t++;
    }
    printf("mcm e' uguale %d", a*t);
}
```

oppure con il costrutto ωN

```
int main()
{
    int a, b, mcm, t=1;
    printf("Inserisci i numeri\n");
    scanf("%d %d",&a, &b);
    while(1){
        mcm = a*t;
        if(mcm%b==0)
            break;
        t++;
    }
}
```

```
printf("L'mcm è %d",mcm);  
return 0;  
}
```

Fibonacci

Scrivere un programma che legge un numero N da tastiera, e stampi i primi N numeri della successione di Fibonacci

es.: N=8

1, 1, 2, 3, 5, 8, 13, 21....

quindi $n_1 = 1$, $n_2 = 1$, $n_3 = n_2 + n_1$, ..., $n_x = n_{x-1} + n_{x-2}$

```
void main() {  
    int max;  
    int n1, n2, s, c=0;  
    printf("Inserisci quanti numeri della successione di fibonacci");  
    scanf("%d",&max);  
    n2 = 0;  
    n1 = 1;  
    while (c++ < max) {  
        printf("%d ", n1);  
        s = n1 + n2;  
        n2 = n1;  
        n1 = s;  
    }  
}
```

Test primalità

Dato un numero N testare se sia un numero primo,
ovvero se non ha divisori tranne 1 e se stesso

```
int main()  
{  
    int cont;  
    int n;  
    printf("inserisci un numero \n");  
    scanf("%d",&n);  
    for(cont=2;cont<=n/2;cont++)  
        if(n%cont==0)  
            break;  
    if(n%cont==0 && n!=2){  
        printf("il numero non è primo");  
    }  
    else{
```

```
        printf("il numero è primo");
    }
}
```

MCD Banale

Dati due numeri, si sceglie il più piccolo dei due. Partendo dalla metà a ritroso trovo il valore che risulta multiplo dei due numeri.

```
int main()
{
    int a, b , minore;
    printf("inserisci il primo numero");
    scanf("%d" , &a);
    printf("inserisci il secondo numero");
    scanf("%d" , &b);
    minore = (a<b)? a:b;
    while(!(a%minore==0 && b%minore==0)){
        minore--;
    }
    printf("l'mcd è %d" , minore);
    return 0;
}
```

MCD Euclide

```
int main()
{
    int X = 21, Y = 7, R;
    do {
        R=X % Y;
        if (R > 0) {
            X = Y;
            Y = R;
        }
    } while (R>0);
    printf (" l' MCD è %d", Y);
    return 0;
}
```

Radice quadrata metodo babilonese

Si vuole estrarre la radice di N a meno di una tolleranza $\epsilon = (0.00001)$

Pongo inizialmente un valore di tentativo X_0 pari a 1.

Calcolo un valore di tentativo successivo calcolato $X_1 = \frac{x_0 + \frac{N}{x_0}}{2}$

se $X_1 - X_0 < \epsilon$ allora X_1 è la radice

altrimenti $X_0 = X_1$ e ricomincio il procedimento

```
int main(){
    double N, X0, X1, epsilon = 1e-10, delta;
    printf("inserisci un numero positivo.\n");
    scanf(" %lf",&N);
    if (N < 0) {
        printf("il numero deve essere positivo.\n");
        return 0;
    }
    X0 = 1.0;
    while(1) {
        X1 = (X0 + N / X0) / 2;
        delta=fabs((X1 - X0));
        printf("X1 = %lf X0= %lf delta=%lf\n", X1, X0,delta);
        if (delta < epsilon)
            break;
        X0=X1;
    }

    printf("la radice quadrata è %lf vs %lf", X1, sqrt(N));

    return 0;
}
```